

Gradient Decoupling for Inductive Temporal Link Prediction: A Controlled Study of Stateful Representations

Duong Viet Hoang^{a,*}, Duong Viet Huy^b, Lun-Min Shih^a

^a*Department of Computer Science, Da-Yeh University
No. 168, University Rd., Dacun, Changhua 515006, Taiwan*

^b*Department of Computer Science, Kent State University
P.O. Box 5190, Kent, OH 44242-0001, United States*

*Corresponding author: Hoangduong4316@icloud.com

ABSTRACT

Inductive temporal link prediction requires a model to score interactions involving entities absent from earlier observations. Jointly optimizing a temporal representation and its link readout is a natural design choice, but it need not be the most effective choice for a particular inductive protocol. This study investigates gradient routing in SR-GNN, a temporal model combining event encoding, recurrent node memory, deterministic pair-history signals and a hierarchical readout. We specify the scored gradient boundary, distinguish parameter learning from event-state evolution, and compare coupled and decoupled versions of the same registered readout. A separate freeze-then-probe procedure examines a different training trajectory. Experiments use chronological partitions of CoEdit, Wikipedia and MOOC and a fixed random-destination evaluation procedure. Checksum-bound results are reported as mean inductive average precision and sample standard deviation across the registered seeds. The observed decoupled arm has higher mean inductive average precision than the coupled arm on each corpus. This supports a bounded within-protocol comparison rather than a general claim that end-to-end learning is harmful. The analysis identifies important interpretation limits: CoEdit shares its source with Wikipedia, its event construction includes retrospective information, the candidate sampler is not destination-type constrained, and temporal memory follows mixed batch and sequential semantics. Retrospective experiments additionally examine design presets, predictor and gate ablations, harder candidate pools, backbone removal and frozen probes. They show that predictor choice and learned features matter in the recorded configurations, while remaining separate from the current same-readout comparison. The study contributes an explicit account of gradient routing and temporal evaluation that makes the observed comparison reproducible and its scope assessable.

Key words: temporal graphs, inductive link prediction, gradient decoupling, recurrent memory, reproducible evaluation

歸納式時序連結預測中的梯度解耦： 具狀態表徵的受控研究

Duong Viet Hoang^{a,*}, Duong Viet Huy^b, Lun-Min Shih^a

^a 大葉大學

515006 臺灣彰化縣大村鄉學府路 168 號

^b 肯特州立大學

P.O. Box 5190, Kent, OH 44242-0001, United States

* 通訊作者：Hoangduong4316@icloud.com

摘要

歸納式時序連結預測需要處理先前觀測中未出現的節點。本研究以結合事件編碼、循環節點記憶、確定性節點對歷史訊號與階層讀出器的 SR-GNN，探討評分路徑的梯度是否應更新時序表徵。我們明確區分梯度邊界、參數學習與事件狀態演化，並在相同已註冊讀出器下比較耦合及解耦訓練；另以先訓練再凍結並重新訓練預測頭的方法作為不同訓練流程的對照。研究採用 CoEdit、Wikipedia 與 MOOC 的時間切分及固定隨機負樣本協定。具校驗碼的封存結果顯示，解耦組在各語料上的平均歸納式平均精確率均高於耦合組。此結果支持協定內的觀察比較，而非端到端學習普遍有害的結論。CoEdit 與 Wikipedia 共享來源，且其事件建構包含事後資訊；候選取樣未限制目的端節點類型，記憶更新亦混合批次與循序語意。此外，回溯實驗涵蓋設計設定、預測器與閘門消融、較難候選池、骨幹移除及凍結探測，顯示預測器選擇與學習表徵在已記錄設定中的作用；各組依設定及資料版本分別呈現，不與目前結果合併。本研究提供可重現的梯度路由及時序評估說明，並界定效能比較的適用範圍。

關鍵詞：時序圖，歸納式連結預測，梯度解耦，循環記憶，可重現評估

I. Introduction

Temporal graphs describe relationships through ordered interactions rather than a single static adjacency matrix. An edit on a collaborative platform or an action in an online course supplies information about both the participating entities and their recent activity. Predicting an interaction therefore involves a representation of evolving history, a candidate-scoring rule, and a decision about which historical information is available at the prediction time. These components jointly define the learning problem. A change to event ordering or candidate construction can alter the meaning of a reported score even when the neural architecture is unchanged.

Inductive prediction adds another constraint: some evaluated interactions involve entities absent from the training population. A model cannot rely solely on well-trained representations of those entities. Shared encoders, temporal neighborhoods, pair statistics and initialization rules become relevant to transfer. Nevertheless, inductive evaluation is not a unique protocol. The treatment of validation entities, the handling of preceding test events and the membership of negative-candidate pools all determine what a new entity means operationally. We state these choices explicitly instead of using the word inductive as a sufficient description of the experiment.

Most task-trained representations allow prediction gradients to update the encoder that produces their features. This gives the encoder a direct way to preserve information useful for the training objective. It can also create a dependence on the training population and on the particular candidate-sampling distribution. Stopping a scored gradient changes this optimization pressure while retaining the representation’s forward value. The resulting tradeoff is empirical: reducing task adaptation can either preserve transferable information or remove a useful learning signal. Neither outcome follows from the stop-gradient operation alone.

Temporal methods make this question especially interesting because they combine learned weights and evolving state. A detached representation is not necessarily static. Node memories can change during forward execution, pair statistics can accumulate observations, and auxiliary objectives can update parameters outside the detached scoring path. Calling the entire model frozen would conflate these mechanisms. Conversely, freezing an encoder after pretraining does not recreate the trajectory obtained by preventing a scored gradient from reaching it throughout initial training.

We study this distinction in SR-GNN, the implementation provided by the accompanying research repository [10]. Its temporal representation combines an event encoder, recurrent node memory and a bank of pair-history signals. A hierarchical readout produces the scored link output. The primary experiment changes whether that readout receives detached features while preserving the registered readout configuration. A sepa-

rate freeze-then-probe arm uses pretraining followed by a newly initialized prediction head. These are deliberately interpreted as different experimental procedures.

We ask whether the same temporal readout generalizes differently when its scored features are connected to or detached from the backbone during training. To interpret that comparison, we specify the derivative boundary and the event-state updates separately. The experiments then compare the registered profiles using matched evaluation conventions. The resulting claim concerns this intervention in SR-GNN; extending it to other architectures or candidate populations requires further experiments.

II. Related Work

1. Temporal representations and link prediction

Inductive graph models use shared functions to represent previously unseen nodes, as illustrated by GraphSAGE [16]. Link prediction also depends on the information supplied to the decoder: SEAL, for example, learns from the local subgraph around a candidate link [40]. In temporal graphs, both the representation and candidate context depend on the order of observations [18, 22].

Event-based architectures differ in how they retain this history. DyRep couples evolving representations with event occurrence [35]; JODIE models user and item trajectories [26]; Temporal Graph Networks organize computation around messages and node memory [32]. Their recurrent components build on gated sequence models [7], but the schedule of memory reads and writes remains an implementation choice. TGAT uses time-aware neighborhood attention [38], while causal anonymous walks capture temporally ordered structural patterns [36]. Graph-Mixer [8], NAT [28] and DyGFormer [39] provide alternatives based on interaction histories and neighborhood context. These architectural differences prevent a routing effect measured in one implementation from being assumed in another.

Our model combines learned event features and recurrent memory with deterministic pair histories. Its intensity-like channel follows a self-exciting recurrence inspired by Hawkes processes [17], and its running moments use Welford’s update [37]. We specify the actual inputs to these recurrences because source-node staleness and pair-specific inter-arrival time are different quantities. The former is used here.

2. Gradient routing and representation probes

A separately trained classifier can assess information available in a learned representation [1]. Kumar et al. show that fine-tuning can distort pretrained features and underperform a linear probe under distribution shift [25]. This motivates examining task adaptation, although their setting differs from an evolving temporal memory. Stop-gradient also appears in BYOL [14] and SimSiam [6], where it forms part of a self-supervised learning procedure. Those results do not predict the outcome of detach-

ing a supervised temporal-link readout.

The information bottleneck [34] and its deep variational formulation [2] motivate regularizing a representation while preserving predictive information. Such an interpretation needs care for deterministic neural representations [3]. In this study, a KL penalty is an optimization term; we do not measure removal of identity information. The main experiment instead compares the same scored readout with connected or detached features. Freeze-then-probe is a separate procedure because its representation has already followed a different training trajectory.

3. Candidate sampling and evaluation

Poursafaei et al. demonstrate the importance of negative-candidate sampling and introduce EdgeBank as a memorization-based comparator [31]. TGB standardizes temporal datasets and evaluation [20], while DyGLib supports comparison of dynamic graph models under shared procedures [39]. These works motivate reporting the candidate population, novelty boundary and history available at scoring time alongside AP.

Model selection and run-to-run variability also affect reported comparisons [5, 4]. We report sample deviations over the retained seeds, rather than treating them as uncertainty over new graph populations. Leakage analysis requires checking which inputs would be available in the claimed prediction setting [21]. This matters for CoEdit’s retrospective construction. Likewise, a favorable inductive score alone cannot demonstrate that identity-related shortcuts have been removed [13]; that explanation would require direct feature measurements and interventions.

Evaluation studies in graph learning and recommendation show that model rankings depend on the training protocol and the strength of tuned comparators [33, 11]. This is relevant to the historical TGAT proxy: reproducing its saved score establishes the arithmetic of that local experiment, but not parity with the published architecture. A benchmark claim would additionally require the intended model, a comparable tuning budget and a common evaluator.

III. Problem and Model Specification

1. Events, candidates and information

Let an event stream contain tuples $e_i = (u_i, v_i, t_i, x_i)$ in non-decreasing timestamp order. The vector x_i contains attributes supplied by the data pipeline. Let $\mathcal{H}_i$ denote the history retained by the particular training or evaluation pass immediately before scoring event i . It need not equal the full raw stream prefix: filtering an evaluation stream before execution changes the observations that enter its state. The model assigns a score to a candidate pair conditioned on this retained history and on the supplied attributes.

We write the temporal representation and output as

$$\begin{aligned} h_i &= F_\theta(u_i, v_i, t_i, x_i; \mathcal{H}_i), \\ p_i &= \sigma(G_\psi(h_i, \phi_i)), \end{aligned} \quad (1)$$

where θ and ψ denote backbone and readout parameters, respectively, and ϕ_i denotes the available pair-history context. This decomposition names the scored boundary; it is not an assertion that every auxiliary quantity is independent of every parameter in F_θ . In the implementation, candidate-specific positive and negative branches have corresponding representations and histories.

2. Deterministic pair-history updates

For an ordered pair, let g_k be the gap supplied to its deterministic update at its k th retained observation. Denote the running mean by μ_k and the sum of squared deviations by M_k . The online moment recurrence is

$$\begin{aligned} \delta_k &= g_k - \mu_{k-1}, \\ \mu_k &= \mu_{k-1} + \delta_k/k, \\ M_k &= M_{k-1} + \delta_k(g_k - \mu_k). \end{aligned} \quad (2)$$

The feature variance is formed from the accumulated moments using the implementation’s population-moment convention. This within-pair feature variance must not be confused with the sample standard deviation across independent training seeds used in the result table. The two quantities answer different questions and have different denominators.

In the current forward implementation, the supplied gap is source-node staleness from the node-memory store. It is not necessarily the elapsed time since the same pair last interacted. The statistics are indexed by pair but accumulate this supplied gap. This distinction limits a literal interpretation as inter-arrival moments of a pair-specific point process.

The recurrence and intensity-like updates can be expressed as

$$r_k = a + (1 - a)r_{k-1} \exp(-\eta g_k), \quad (3)$$

$$\ell_k = \mu_0 + (\ell_{k-1} - \mu_0) \exp(-\beta g_k) + \alpha. \quad (4)$$

Here r_k summarizes recent recurrence, ℓ_k is the intensity-like channel, and the constants govern decay and event increments. Equation (4) includes the baseline and event increment. Omitting either would describe a different update law. The resulting quantity is used as model context rather than reported as a calibrated event probability.

Fast and slow exponentially weighted rate estimates summarize changes in cadence. With $q_k = 1/\max(g_k, \epsilon)$, their generic form is

$$\begin{aligned} f_k &= a_f q_k + (1 - a_f) f_{k-1}, \\ s_k &= a_s q_k + (1 - a_s) s_{k-1}. \end{aligned} \quad (5)$$

The gap floor keeps coincident observations finite. Relative fast and slow rates and a decaying rate peak provide additional context for the readout. Their values depend on time units and update ordering. These conventions must therefore travel with a result rather than being inferred from a feature name.

The model also maintains learned node-memory and pair-representation components. Deterministic statistics offer a direct summary of observations, whereas learned transformations can adapt how the summaries and event attributes are combined. Decoupling the scored path changes the latter adaptation without deleting the former information. This is why an effective detached arm does not imply that a random or history-free encoder would be equally effective.

3. Readout and gradient boundary

Let $D_0(h) = h$ and $D_1(h) = \text{sg}(h)$, where stop-gradient is defined by its forward value and backward derivative:

$$\text{sg}(h) = h, \quad \frac{\partial \text{sg}(h)}{\partial h} = 0. \quad (6)$$

The primary arms use the same registered readout family and differ in whether its scored feature input passes through D_0 or D_1 :

$$p_i^{(d)} = \sigma(G_\psi(D_d(h_i), \phi_i)), \quad d \in \{0, 1\}. \quad (7)$$

This boundary is applied to both positive and negative scoring branches. Detaching only observed interactions would leave a task-gradient route through the sampled negatives and would implement a different intervention.

For the path through h_i , the chain rule gives

$$\nabla_{\theta} L_{\text{score}}|_{h_i} = \frac{\partial L_{\text{score}}}{\partial G} \frac{\partial G}{\partial D_d} \frac{\partial D_d}{\partial h_i} \frac{\partial h_i}{\partial \theta}. \quad (8)$$

That contribution vanishes for the detached branch. This is a path-specific computational statement. It does not establish that every route from the complete weighted training objective to every backbone parameter vanishes. Additional dependencies through weights, shared modules or auxiliary losses must be inspected separately. In particular, the current soft-compliance weighting should not be silently treated as a detached constant when analyzing the entire objective.

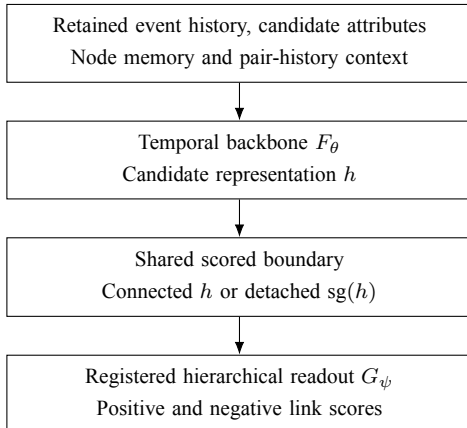


Figure 1. Scored feature path in the primary comparison.

Figure 1 identifies the boundary changed by the experiment. Auxiliary objectives and deterministic store writes are not drawn as scoring edges. Their omission from this diagram is a visual simplification, not a claim that they are absent from training. The forward representations of arms with identical parameters would agree at the boundary, but their learned trajectories need not agree after optimization.

4. Training objective and auxiliary routes

For a scored logit z and label y , the binary logistic loss is

$$\ell(z, y) = -y \log \sigma(z) - (1 - y) \log(1 - \sigma(z)). \quad (9)$$

The implementation combines weighted per-example prediction losses with enabled auxiliary terms. A compact description is

$$L = \frac{1}{|\mathcal{B}|} \sum_{j \in \mathcal{B}} w_j \ell(z_j, y_j) + \lambda_{\text{KL}} L_{\text{KL}} + \sum_{a \in \mathcal{A}} \lambda_a L_a. \quad (10)$$

The auxiliary collection includes the active violation, transition, edge-state and entropy-related terms selected by the resolved profile. Disabled terms contribute nothing. The code, protocol and resolved configuration specify their actual inclusion and weights. Equation (10) is a dependency summary rather than a substitute for that operational specification.

Variational autoencoding uses a KL term within a specified latent-variable objective [24]. Connecting such regularization to information retained by an encoder requires identifying the distributions and approximation being optimized. Poole et al. analyze bias–variance tradeoffs among variational mutual-information bounds [30]. Here, the recorded objective does not provide an empirical estimate of identity information. A lower auxiliary loss would therefore not, by itself, establish that identity cues were removed or explain the observed AP difference. We use the objective to specify which parameters can receive gradients and leave the proposed statistical mechanism open.

5. State updates and batching

Scoring and store mutation must be ordered consistently for positive and negative candidates. A positive candidate reads pre-update pair history; a negative candidate reads its own available pair history without inserting a fictional observation. Actual observations subsequently update the stores. If positives instead received their post-event counts while negatives saw only pre-event counts, the model could exploit an evaluation artifact rather than historical predictive information.

Repeated pairs within a batch require additional care. A single batch snapshot would expose the same old pair statistics to all repeated occurrences. The causal-batch option folds deterministic pair accumulators in stable event order so a later occurrence can read the state following an earlier occurrence in that batch. This addresses a store-level timing issue. It should not be interpreted as sequential execution of the entire neural model.

Recurrent node memory follows a different registered convention. Candidate updates are computed from a batch snapshot, and collisions on a node index are resolved by committing the final candidate in stable input order. Thus the procedure combines sequential deterministic pair updates with batch-snapshot neural memory. This is a reproducible approximation whose batch size is part of the protocol. A comparison to fully event-sequential neural updates would require a separate experiment.

6. Freeze-then-probe procedure

The freeze-then-probe arm pretrains with the separate main prediction head, selects a validation checkpoint, freezes representation parameters and reinitializes the main head. The probe phase then trains the new head while forward event processing continues to update temporal stores. Its own validation selection determines the evaluated checkpoint. Freezing weights therefore differs from freezing a precomputed feature matrix.

IV. Data and Experimental Protocol

1. Corpus construction and identities

Wikipedia and MOOC are obtained from the public JODIE data sources [26]. They are bipartite streams whose endpoint roles have separate node-ID namespaces in the current preprocessing. The distinction prevents a user and an item with the same original integer label from sharing a memory row. CoEdit instead represents contributor-to-contributor interactions in a homogeneous namespace. Topology is a property of each registered corpus and is not inferred from whether source and destination integers happen to overlap.

CoEdit is derived from Wikipedia edits. For each page, the builder considers edits by distinct contributors within a 60-minute window. It emits a contributor pair with the average of the edit attributes and the midpoint of their timestamps. It retains the first 80,000 qualifying pairs in page first-observation order and then applies stable chronological ordering. This procedure is not equivalent to taking the earliest 80,000 qualifying pairs globally. The selection order may influence which pages and recurrence patterns are represented.

The midpoint convention also limits a forecasting interpretation. The later edit is required to construct a co-edit event, yet its attributes enter a derived event timestamped before that later edit. Thus chronological ordering of the derived stream does not make all its attributes prospectively available at the midpoint. The present experiment is a comparison on this registered derived corpus. A deployment-oriented forecasting study should timestamp a constructed interaction at its actual availability time or exclude attributes unavailable at the proposed decision time.

CoEdit and Wikipedia consequently do not provide independent source replications. They offer different graph constructions from a shared source, whereas MOOC supplies a separate in-

teraction source. This distinction matters when assessing how broadly an observed ordering generalizes. An effect repeated across these corpora is more informative than an effect on a single graph construction, but it is not evidence from an equal number of independent data-generating environments.

2. Chronological partitions and inductive population

The registered split assigns the chronological event stream to training, validation and testing in proportions 70/15/15. The inductive node set contains nodes occurring in the test partition but in neither training nor validation. An inductive test event touches at least one such node. This definition makes the novelty boundary later than a definition that treats every node absent from training as unseen.

The evaluator filters the test events before executing the inductive pass. As a result, a non-inductive test event omitted by that filter does not update the inductive evaluator’s memory, even if it precedes a retained event in the raw stream. Training and validation history are replayed to initialize the evaluation state. This filtered-stream convention should be distinguished from replaying every test event and calculating the metric only on an inductive subset. The latter exposes the model to a different retained history.

New-node initialization is part of the model behavior rather than a separate source of supervision. Shared functions can score a candidate with a new endpoint, but the memory available for that endpoint depends on which prior retained events touched it. An interaction involving an inductive node is not necessarily its first evaluated interaction. Repeated appearances within the test period can therefore contribute useful history even in the inductive metric.

3. Candidate sampling

For each observed interaction the evaluator constructs a sampled negative while retaining the source. The destination pool follows the seen or unseen status of the true destination. A sampled destination identical to the true one is rejected. This does not amount to rejecting every true interaction at the same timestamp, and it does not ensure that the replacement endpoint has the destination role required by a bipartite application.

The same evaluation sampling convention is used across primary arms, with reset evaluation randomness supporting matched draws. This makes the internal contrast more interpretable than a comparison in which each arm sees different candidates. It still leaves a limitation shared by all arms: a candidate pool can be easier than the application population. Near-ceiling performance must be interpreted in that context rather than as evidence that arbitrary future links are predictable.

Negative candidates receive their own historical pair context and do not write themselves into memory as observed events. Reusing the positive pair’s history for a negative would erase the

candidate distinction, while forcing every negative history to be empty would introduce another asymmetry. The implemented read-only lookup avoids these specific mistakes. It does not settle all potential leakage or candidate-validity questions.

4. Optimization and model selection

Table 1 summarizes the registered training configuration. The primary arms use the same backbone size, batch size, optimizer family and schedule. Adam is used for optimization [23] in PyTorch [29]. The registered optimizer uses its weight-decay setting; it is not replaced by AdamW, whose decoupled decay changes the update rule [27]. Validation-based model selection takes place within the registered training procedure; the test endpoint is not used to select an epoch. The freeze-then-probe profile includes an additional probe phase and must not be described as having the same optimization trajectory as the primary arms.

Table 1. Registered experimental settings.

Setting	Registered value
Corpora	CoEdit, Wikipedia, MOOC
Seeds	1, 7, 42
Chronological split	70/15/15
Training epochs	20
Probe epochs	20 after pretraining
Hidden dimension	128
Batch size	500
Learning rate	0.001
Weight decay	0.00001
Optimizer and schedule	Adam; cosine annealing
Primary readout	Shared hierarchical readout
Scored feature routing	Connected or detached
Compliance	Soft; floor 0.05
Negative regime	Fair random destination

The runtime warmup is state-neutral. Although a real training pass initializes the computational kernels, parameters, buffers, optimizer state, random streams and temporal stores are restored afterwards. This prevents initialization work from becoming hidden extra optimization. The runner also checks finite values and records failures rather than replacing invalid values or silently dropping failed tasks from an aggregate.

5. Metric and uncertainty

Average precision summarizes precision at increasing recall levels on the evaluated positive and negative scores. For recall thresholds indexed by k , it can be written as

$$AP = \sum_k (R_k - R_{k-1}) P_k, \quad (11)$$

where P_k and R_k are precision and recall at the corresponding threshold. The experiment uses the reference implementation recorded by the runner. Average precision is conditional on the sampled candidates and their label balance; it is not a calibrated probability that a predicted interaction will occur in deployment.

For selected seed values $a_1, \dots, a_n$, the reporting convention is

$$\bar{a} = \frac{1}{n} \sum_{j=1}^n a_j, \quad s = \sqrt{\frac{\sum_{j=1}^n (a_j - \bar{a})^2}{n-1}}. \quad (12)$$

The reported standard deviation describes variation across the retained training seeds. It is not a confidence interval for performance on new graphs, an uncertainty interval over all possible test populations, or a measure of temporal dependence between individual test events. We do not infer significance from overlap or non-overlap of these descriptive deviations.

A paired effect would be computed from matched per-seed differences, not by subtracting standard deviations from separate arms. Establishing a confidence interval for such an effect requires its own statistical specification and sufficient supporting data. The principal statement here is the observed ordering of the registered means. It does not require or imply a claim that every seed favors the same arm.

V. Results

1. Current within-protocol comparison

Table 2 reports the current registered inductive average precision. Values are rounded to four decimal places from the checksum-bound matrix. Every row contains the same registered seed count. The connected and detached profiles use the shared scored readout configuration, so the primary comparison is not a replacement of that readout by a different predictor.

Table 2. Inductive AP: mean and sample SD.

Corpus	Profile	AP
CoEdit	Coupled	0.9787 $\pm$ 0.0021
	Decoupled	0.9891 $\pm$ 0.0016
	Freeze then probe	0.8381 $\pm$ 0.0484
MOOC	Coupled	0.9910 $\pm$ 0.0047
	Decoupled	0.9947 $\pm$ 0.0007
	Freeze then probe	0.9862 $\pm$ 0.0009
Wikipedia	Coupled	0.9967 $\pm$ 0.0003
	Decoupled	0.9983 $\pm$ 0.0003
	Freeze then probe	0.9623 $\pm$ 0.0128

Under the registered current-corpus protocol, the observed decoupled SR-GNN arm has higher mean inductive average precision than the coupled end-to-end arm on each registered corpus. The observed differences apply to the shared-readout configurations and candidate population specified above.

The probe follows a different pretraining and head-retraining trajectory. Its lower mean AP can reflect the pretrained features, head capacity or optimization budget; it does not establish irreversible representation damage.

The current table uses the selected completed runs from the frozen matrix. Failed and superseded attempts remain in the ac-

companying execution ledger and are excluded from these aggregates. All displayed means and sample deviations are reconstructed from per-seed records; no new training was performed for this manuscript revision.

2. Retrospective experiments from model development

The earlier study investigated a broader set of configurations than the shared-readout comparison. We retain those experiments here to examine predictor choice, candidate-pool sensitivity, learned-backbone contributions and recovery by a newly trained probe. Tables 3–7 reconstruct means and sample deviations directly from recovered per-seed records. The accompanying source map identifies each input file, checksum, selected observations and aggregation rule. All tables report mean AP and sample SD. The preset comparison uses five seeds; the remaining historical comparisons use three seeds per configuration. The backbone table retains six decimals to show its small nonzero Wikipedia deviation.

These are retrospective results. The recovered records describe chronological 70/15/15 partitions, a 20-epoch training budget and average-precision evaluation, but do not bind every run to its original data, source and environment as completely as the current matrix does. We consequently interpret each experimental family within its recorded configuration and do not pool it with Table 2. Separate repetitions of configuration B remain separate rather than selecting whichever repetition gives the largest contrast.

3. Preset comparison and predictor ablations

The five-seed CoEdit experiment in Table 3 recovers the earlier draft’s broad preset comparison. B uses the detached hierarchical scoring configuration, whereas C selects the alternative end-to-end design preset. The local TGAT-labeled implementation is a simplified proxy; its result describes that implementation and does not establish a comparison against a faithful reproduction of published TGAT.

Table 3. CoEdit preset comparison.

Configuration	Inductive AP	Transductive AP
B preset	0.9876 ± 0.0030	0.9985 ± 0.0003
C preset	0.7593 ± 0.0140	0.9600 ± 0.0044
TGAT proxy	0.8466 ± 0.0133	0.8645 ± 0.0083

The B–C difference is much larger for inductive AP than for transductive AP. This observation motivated the routing question, but it cannot identify a gradient effect on its own: the presets change predictor and gating choices together. The current shared-readout experiment addresses a narrower question by preserving the readout while changing the scored-feature derivative path. The large historical preset gap should therefore not be used as its effect size.

Table 4 separates the preset changes. Relative to B, K1 enables the main predictor, K2 changes the compliance gate from soft to hard, and K3 sets the floor to zero. K2+K3 combines the gate changes, while C combines all three. The CoEdit study covers the full set; the Wikipedia and MOOC studies retain the B–K1 contrast.

Table 4. Predictor and gate ablations.

Corpus	Configuration	AP
CoEdit	B	0.9899 ± 0.0016
	K1: predictor	0.7788 ± 0.0193
	K2: hard gate	0.9868 ± 0.0014
	K3: zero floor	0.9889 ± 0.0018
	K2 + K3	0.9870 ± 0.0021
	C: K1 + K2 + K3	0.7798 ± 0.0221
Wikipedia	B	0.9981 ± 0.0002
	K1: predictor	0.9490 ± 0.0072
MOOC	B	0.9935 ± 0.0008
	K1: predictor	0.9844 ± 0.0017

On CoEdit, enabling the main predictor produces the largest observed reduction, whereas the gate-only variants remain close to B. The combined C preset also lies near K1. The corresponding B–K1 ordering appears on Wikipedia and MOOC, with smaller differences. This pattern points to predictor choice as an important part of the earlier result. It does not establish that gate changes are universally irrelevant: only the recorded settings and budgets were tested, and interactions can depend on the readout. In particular, K1 is not the same intervention as reconnecting the scored features of an otherwise unchanged hierarchical head.

4. Sensitivity to harder candidate pools

Table 5 restores the harder-negative experiment. Each trained model is evaluated under three candidate-pool rules; these evaluations are repeated measurements of that model, not independent retrains. The records specify a fixed evaluation seed for pairing B and K1.

The sampler labels require a precise interpretation. In the accompanying implementation, the historical pool is a multiset of training destinations, and the inductive pool is a multiset of destinations belonging to test pairs absent from training. Candidates are drawn from these destination pools and combined with the evaluated source. Thus, a historical candidate need not be a previously observed partner of that particular source, and inductive here does not mean exclusively unseen nodes. The legacy field named `indpool_from_train` denotes a nonempty test-only pool in the corrected implementation. Its name should not be read as the pool definition.

Table 5. Candidate-pool sensitivity.

Corpus	Pool	B	K1
CoEdit	Random	0.9898 ± 0.0012	0.7899 ± 0.0545
	Historical	0.9573 ± 0.0031	0.5294 ± 0.0276
	Inductive	0.9625 ± 0.0024	0.5579 ± 0.0215
Wikipedia	Random	0.9968 ± 0.0005	0.9011 ± 0.0106
	Historical	0.9744 ± 0.0047	0.5585 ± 0.0147
	Inductive	0.9682 ± 0.0035	0.5423 ± 0.0032

Both configurations lose AP under the harder pools, but K1 loses substantially more. B retains the higher mean in every reported corpus–pool combination. This extends the descriptive B–K1 ordering beyond the random-destination setting used in those records. It does not establish equivalent behavior for the current same-readout detach contrast, which was not evaluated under these archived pools.

There are further limits to this sensitivity test. The inductive pool uses test-split membership to construct an offline evaluation population. The rejection loop checks against positive pairs from the evaluation split but stops after a bounded number of retries, so it does not guarantee that every sampled negative is valid. The retained summaries do not count residual collisions. Accordingly, these results characterize the implemented pool perturbations, rather than certify benchmark parity or a fully causal online candidate-generation policy.

5. Contribution of the learned backbone

The backbone-removal experiment asks whether deterministic pair histories alone explain the performance of B. The reduced arm bypasses the event encoder and recurrent-memory contribution, keeps the memory unchanged, and supplies zero learned edge features to the scoring head. Pair-history channels remain available. The scoring head is still trainable, so this is a deterministic-feature representation with a learned readout, not a globally parameter-free model.

Table 6. Learned-backbone contribution.

Corpus	Representation	Inductive AP
CoEdit	Full B	0.988326 ± 0.002338
	Pair statistics	0.920500 ± 0.001668
Wikipedia	Full B	0.996296 ± 0.000872
	Pair statistics	0.896506 ± 0.000004

The full representation has higher mean AP on both corpora. At the same time, the reduced representation retains substantial predictive performance, consistent with pair histories providing useful information. Together these observations favor a combined account: deterministic channels contribute useful signals, while the learned backbone adds information in these recorded configurations. Removing multiple components at once does not isolate the contribution of the event encoder, recurrent memory

or auxiliary objective individually. A factorial study would be needed to separate those contributions.

6. Historical freeze-then-probe experiments

The recovered probe records compare a decoupled training trajectory with a freeze-then-probe trajectory. Table 7 reports the two procedures separately for CoEdit and Wikipedia. It also preserves an explicitly ID-corrected Wikipedia probe record as a distinct version. That record contains all three seeds; it is not merged with the earlier Wikipedia probe or treated as a matched correction experiment without a version-matched control.

Table 7. Freeze-then-probe comparison.

Corpus	Procedure	Inductive AP
CoEdit	Decoupled	0.9883 ± 0.0028
	Freeze then probe	0.7684 ± 0.0064
Wikipedia	Decoupled	0.9961 ± 0.0014
	Freeze then probe	0.9027 ± 0.0177
Wikipedia	Probe, ID-corrected	0.9513 ± 0.0030

In the earlier records, the probe remains below the decoupled procedure on both corpora. The ID-corrected Wikipedia probe has a different endpoint from the earlier probe, illustrating why dataset and configuration identity matter to this comparison. Neither contrast demonstrates irreversible representation damage. A finite probe budget tests a particular recovery procedure, and differences can reflect the pretrained features, head capacity, optimization or data version. The current probe results in Table 2 provide a separately specified comparison and should retain their own interpretation.

VI. Discussion

1. What the routing contrast can explain

The mean ordering indicates that scored-feature routing is an empirically relevant design choice in this registered implementation. Several mechanisms could contribute. Restricting the scored gradient may preserve useful history features; it may reduce adaptation to the sampled training candidates; or it may change the interaction between the scored objective and auxiliary terms. The aggregate endpoint does not distinguish these explanations.

One possible explanation is adaptation to identity-related shortcuts [13]. Direct evidence would require measuring identity-related information, testing its relationship to performance, and intervening on the relevant features while holding other factors stable. A high inductive score and a favorable detached-arm ordering do not by themselves reveal which feature caused that ordering. They are outcomes to be explained, not measurements of the proposed mechanism.

The deterministic channels offer another interpretation. When recurrence and cadence already supply useful signals, preserving them may reduce the need for task-driven transformation.

However, performance could also depend on learned event features and recurrent memory. A complete decomposition would compare a deterministic-only representation, a learned representation without pair statistics, and controlled combinations under the same current protocol. The backbone-removal results in Table 6 show that the recorded full representation outperforms a learned readout over pair statistics alone. They support the usefulness of the combined representation, while leaving the individual learned components and their interaction with the current routing intervention unresolved.

2. Why historical contrasts differ

The earlier development record contains several different notions of coupling. A design-preset contrast changes more than a scored boundary; a main-predictor knob can replace the readout with another head; and a detach toggle on the same head changes a more specific route. Treating all of these as repeated estimates of a single causal effect would conflate architectural and optimization changes.

The current shared-readout experiment makes the narrower comparison explicit. Both primary profiles keep the main-predictor switch off, and the detach setting distinguishes the scored representation path. The historical preset gap therefore measures a broader configuration change than the current routing contrast.

3. Generalization and practical use

The practical implication is to test a gradient-routing boundary when adapting this kind of model to an inductive setting. It is not to prohibit end-to-end training. A different encoder, a different candidate population or an application with weak deterministic history signals could benefit from direct task adaptation. The relevant design choice should be evaluated with the information available at the actual decision point.

For recommendation-like tasks, valid endpoint types and plausible alternatives are especially important. An evaluator drawing candidates across all node roles can reward discrimination that the deployed system never needs to perform. Replacing it with a destination-valid pool changes the task difficulty and may change the relative effect of routing. Such an evaluation should be reported as an extension with its own data and protocol identity.

For prospective forecasting, the temporal availability of event attributes is equally important. In a constructed corpus, an attribute can be attached to a timestamp even though it is known only after another event occurs. Preventing post-score memory writes does not correct that input-level availability issue. Deployment claims therefore require both a causally ordered state procedure and a causally available input construction.

4. Interpreting scores beyond the sampled task

AP evaluates the ordering of positive and sampled-negative scores. Probability calibration asks a different question: whether a predicted probability agrees with the frequency of the event [15]. A model may preserve its ranking while its numerical probabilities change. Consequently, the high AP in the current matrix does not justify applying a probability threshold to an operational stream without checking the event prevalence and candidate-generation policy. That check would require a deployment-specific sample and a separate calibration analysis.

Benchmark design also determines what transfer is being tested. OGB emphasizes standardized graph datasets and evaluators [19]; TGB 2.0 extends temporal evaluation to multi-relational and heterogeneous settings [12]. These settings would test whether the observed routing behavior survives changes in endpoint roles, edge types and historical support. They cannot be represented by adding the present corpora to a larger pooled mean. Each setting needs a valid candidate population and an architecture-specific definition of the detached boundary.

Finally, comparable held-out scores can conceal different behavior under deployment conditions, a problem examined as underspecification [9]. For a stateful model, useful stress tests would vary the retained history, repeated-interaction frequency and time gaps while preserving valid temporal inputs. Such tests could distinguish sensitivity to event memory from sensitivity to pair-history statistics. They would complement the current endpoint comparison rather than turn its small set of training seeds into evidence about all future graph populations.

5. Limitations and next experiments

The study has a limited seed budget and a restricted set of corpora and model profiles. The reported uncertainty concerns selected seed variation rather than sampling of independent environments. CoEdit’s shared origin with Wikipedia and its construction order further constrain external validity. These limits remain even when all recorded hashes and aggregate computations agree.

The primary evaluation uses random destinations. The historical harder-pool results extend the B-K1 configuration comparison, but their pool construction and bounded rejection introduce additional limitations. Further tests should apply destination-valid and strictly checked harder-negative regimes to the current same-readout intervention. They should preserve matched draws across arms, record pool support and avoid silent fallbacks when a requested pool is empty. A failed or degenerate pool should remain visible as an evaluation outcome rather than being interpreted as evidence of robustness.

The intervention should also be replicated within faithful implementations of other temporal architectures. A sound replication must preserve each model’s forward computation and change a well-defined gradient boundary. Replacing a published architecture by a simplified local approximation would answer a dif-

ferent question. Probe extensions should vary head capacity and recovery budget before proposing an explanation based on irreversibility.

Finally, a full gradient audit should examine both scored-output ancestry and the complete weighted objective. It should include candidate branches and auxiliary dependencies, and separate parameter gradients from event-state writes. Such an audit can establish a computational property for a specific configuration. It cannot by itself determine the statistical reason for an observed generalization difference.

VII. Conclusion

This study compares connected and detached scored-feature paths in a registered SR-GNN implementation for inductive temporal link prediction. The observed decoupled arm has higher mean inductive average precision on each registered corpus. The result supports evaluating gradient routing as an explicit design choice within the stated model and candidate-sampling protocol.

The fuller specification distinguishes scored derivatives, auxiliary optimization, recurrent memory and deterministic pair-history updates. It also exposes the role of filtered test history, candidate validity and attribute availability in interpreting a temporal result. The observed probe arm remains a separate training procedure, and historical configurations remain distinguishable from the current shared-readout comparison. These distinctions provide a basis for further experiments without turning a bounded performance observation into a universal or causal claim.

References Cited

- [1] Alain, G. and Y. Bengio (2016) Understanding intermediate layers using linear classifier probes. arXiv:1610.01644.
- [2] Alemi, A. A., I. Fischer, J. V. Dillon and K. Murphy (2017) Deep variational information bottleneck. International Conference on Learning Representations. arXiv:1612.00410.
- [3] Amjad, R. A. and B. C. Geiger (2020) Learning Representations for Neural Network-Based Classification Using the Information Bottleneck Principle. IEEE Transactions on Pattern Analysis and Machine Intelligence, 42, 2225–2239. doi:10.1109/tpami.2019.2909031.
- [4] Bouthillier, X., P. Delaunay, M. Bronzi, A. Trofimov, B. Nychporuk, J. Szeto, et al. (2021) Accounting for Variance in Machine Learning Benchmarks. arXiv:2103.03098.
- [5] Cawley, G. C. and N. L. C. Talbot (2010) On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation. Journal of Machine Learning Research, 11, 2079–2107.
- [6] Chen, X. and K. He (2021) Exploring simple Siamese representation learning. IEEE/CVF Conference on Computer Vision and Pattern Recognition, 15750–15758.
- [7] Cho, K., B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, et al. (2014) Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), 1724–1734. doi:10.3115/v1/d14-1179.
- [8] Cong, W., S. Zhang, J. Kang, B. Yuan, H. Wu, X. Zhou, H. Tong and M. Mahdavi (2023) Do we really need complicated model architectures for temporal networks? International Conference on Learning Representations. arXiv:2302.11636.
- [9] D’Amour, A., K. Heller, D. Moldovan, B. Adlam, B. Alipanahi, A. Beutel, et al. (2020) Underspecification Presents Challenges for Credibility in Modern Machine Learning. arXiv:2011.03395.
- [10] Duong, V. H. and L.-M. Shih (2026) Temporal link decoupling: source, protocol and scientific evidence. Research repository. <https://github.com/hoangduong6210/temporal-link-decoupling>.
- [11] Ferrari Dacrema, M., P. Cremonesi and D. Jannach (2019) Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches. Proceedings of the 13th ACM Conference on Recommender Systems. doi:10.1145/3298689.3347058.
- [12] Gastinger, J., S. Huang, M. Galkin, E. Loghmani, A. Parviz, F. Poursafaei, et al. (2024) TGB 2.0: A Benchmark for Learning on Temporal Knowledge Graphs and Heterogeneous Graphs. arXiv:2406.09639.
- [13] Geirhos, R., J. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, et al. (2020) Shortcut Learning in Deep Neural Networks. arXiv:2004.07780.
- [14] Grill, J., F. Strub, F. Altché, C. Tallec, P. H. Richemond, E. Buchatskaya, et al. (2020) Bootstrap your own latent: A new approach to self-supervised Learning. arXiv:2006.07733.
- [15] Guo, C., G. Pleiss, Y. Sun and K. Q. Weinberger (2017) On Calibration of Modern Neural Networks. arXiv:1706.04599.
- [16] Hamilton, W. L., R. Ying and J. Leskovec (2017) Inductive Representation Learning on Large Graphs. arXiv:1706.02216.
- [17] Hawkes, A. G. (1971) Spectra of some self-exciting and mutually exciting point processes. Biometrika, 58(1), 83–90.
- [18] Holme, P. and J. Saramäki (2012) Temporal Networks. Physics Reports, 519(3), 97–125. doi:10.1016/j.physrep.2012.03.001.

- [19] Hu, W., M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, et al. (2020) Open Graph Benchmark: Datasets for Machine Learning on Graphs. arXiv:2005.00687.
- [20] Huang, S., F. Poursafaei, J. Danovitch, M. Fey, W. Hu, E. Rossi, J. Leskovec, M. Bronstein, G. Rabusseau and R. Rabbany (2023) Temporal graph benchmark for machine learning on temporal graphs. *Advances in Neural Information Processing Systems*, 36. arXiv:2307.01026.
- [21] Kapoor, S. and A. Narayanan (2022) Leakage and the Reproducibility Crisis in ML-based Science. arXiv:2207.07048.
- [22] Kazemi, S. M., R. Goel, K. Jain, I. Kobyzev, A. Sethi, P. Forsyth, et al. (2019) Representation Learning for Dynamic Graphs: A Survey. arXiv:1905.11485.
- [23] Kingma, D. P. and J. Ba (2015) Adam: A method for stochastic optimization. *International Conference on Learning Representations*. arXiv:1412.6980.
- [24] Kingma, D. P. and M. Welling (2014) Auto-encoding variational Bayes. *International Conference on Learning Representations*. arXiv:1312.6114.
- [25] Kumar, A., A. Raghunathan, R. Jones, T. Ma and P. Liang (2022) Fine-tuning can distort pretrained features and underperform out-of-distribution. *International Conference on Learning Representations*. arXiv:2202.10054.
- [26] Kumar, S., X. Zhang and J. Leskovec (2019) Predicting dynamic embedding trajectory in temporal interaction networks. *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1269–1278.
- [27] Loshchilov, I. and F. Hutter (2017) Decoupled Weight Decay Regularization. arXiv:1711.05101.
- [28] Luo, Y. and P. Li (2022) Neighborhood-aware scalable temporal network representation learning. *Proceedings of the First Learning on Graphs Conference*, PMLR, 198, 1:1–1:18.
- [29] Paszke, A., S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, et al. (2019) PyTorch: An Imperative Style, High-Performance Deep Learning Library. arXiv:1912.01703.
- [30] Poole, B., S. Ozair, A. van den Oord, A. A. Alemi and G. Tucker (2019) On Variational Bounds of Mutual Information. arXiv:1905.06922.
- [31] Poursafaei, F., S. Huang, K. Pelrine and R. Rabbany (2022) Towards better evaluation for dynamic link prediction. *Advances in Neural Information Processing Systems*, 35. arXiv:2207.10128.
- [32] Rossi, E., B. Chamberlain, F. Frasca, D. Eynard, F. Monti and M. Bronstein (2020) Temporal graph networks for deep learning on dynamic graphs. *ICML Workshop on Graph Representation Learning and Beyond*. arXiv:2006.10637.
- [33] Shchur, O., M. Mumme, A. Bojchevski and S. Günnemann (2018) Pitfalls of Graph Neural Network Evaluation. arXiv:1811.05868.
- [34] Tishby, N., F. C. Pereira and W. Bialek (1999) The information bottleneck method. *Proceedings of the Allerton Conference on Communication, Control and Computing*. Preprint: arXiv:physics/0004057.
- [35] Trivedi, R., M. Farajtabar, P. Biswal and H. Zha (2019) DyRep: Learning representations over dynamic graphs. *International Conference on Learning Representations*.
- [36] Wang, Y., Y.-Y. Chang, Y. Liu, J. Leskovec and P. Li (2021) Inductive representation learning in temporal networks via causal anonymous walks. *International Conference on Learning Representations*. arXiv:2101.05974.
- [37] Welford, B. P. (1962) Note on a method for calculating corrected sums of squares and products. *Technometrics*, 4(3), 419–420.
- [38] Xu, D., C. Ruan, E. Korpeoglu, S. Kumar and K. Achan (2020) Inductive representation learning on temporal graphs. *International Conference on Learning Representations*. arXiv:2002.07962.
- [39] Yu, L., L. Sun, B. Du and W. Lv (2023) Towards better dynamic graph learning: New architecture and unified library. *Advances in Neural Information Processing Systems*, 36. arXiv:2303.13047.
- [40] Zhang, M. and Y. Chen (2018) Link Prediction Based on Graph Neural Networks. arXiv:1802.09691.